

ENGENHARIA DE SOFTWARE

Felipe Meira

Projeto de Bloco - TP1

Desenvolvimento Back-End

PROFESSOR(A)

Ricardo Frohlich da Silva

DATA DE ENTREGA

11/11/2026

2026

Tarefa 1 – Levantamento dos requisitos

Requisitos Funcionais (RF)

RF-01

Recepcionista (Ator) conseguir cadastrar animais com seus dados e os dados do tutor

RF-02

Recepcionista (Ator) conseguir cadastrar veterinários com nome e CRMV

RF-03

Recepcionista (Ator) conseguir agendar consultas associando animal, veterinário, data e horário

RF-04

Recepcionista (Ator) conseguir cancelar consultas que estejam agendadas

RF-05

Veterinário (Ator) conseguir consultar a agenda do dia

RF-06

Veterinário (Ator) conseguir marcar sua consulta como realizada e registrar observações

Requisitos NÃO Funcionais (RNF)

RNF-01

O programa deve ser compatível com C# e .NET 10

RNF-02

O programa standalone inicial deve executar em ambiente Linux compatível

RNF-03

O sistema deve apresentar mensagens compreensíveis ao rejeitar operações inválidas

RNF-04

Os dados exibidos nas consultas devem ser organizados e compreensíveis para o usuário

RNF-05

O sistema final deve ter uma meta definida de tempo de resposta para operações comuns

Regras de Negócio (RN)

RN-01

A duração de cada consulta é de 30 minutos

RN-02

Agendamentos somente de segunda a sexta-feira, das 8h às 18h

RN-03

Impedir duas consultas agendadas para o mesmo veterinário no mesmo horário

RN-04

Impedir agendamento em datas anteriores à data atual

RN-05

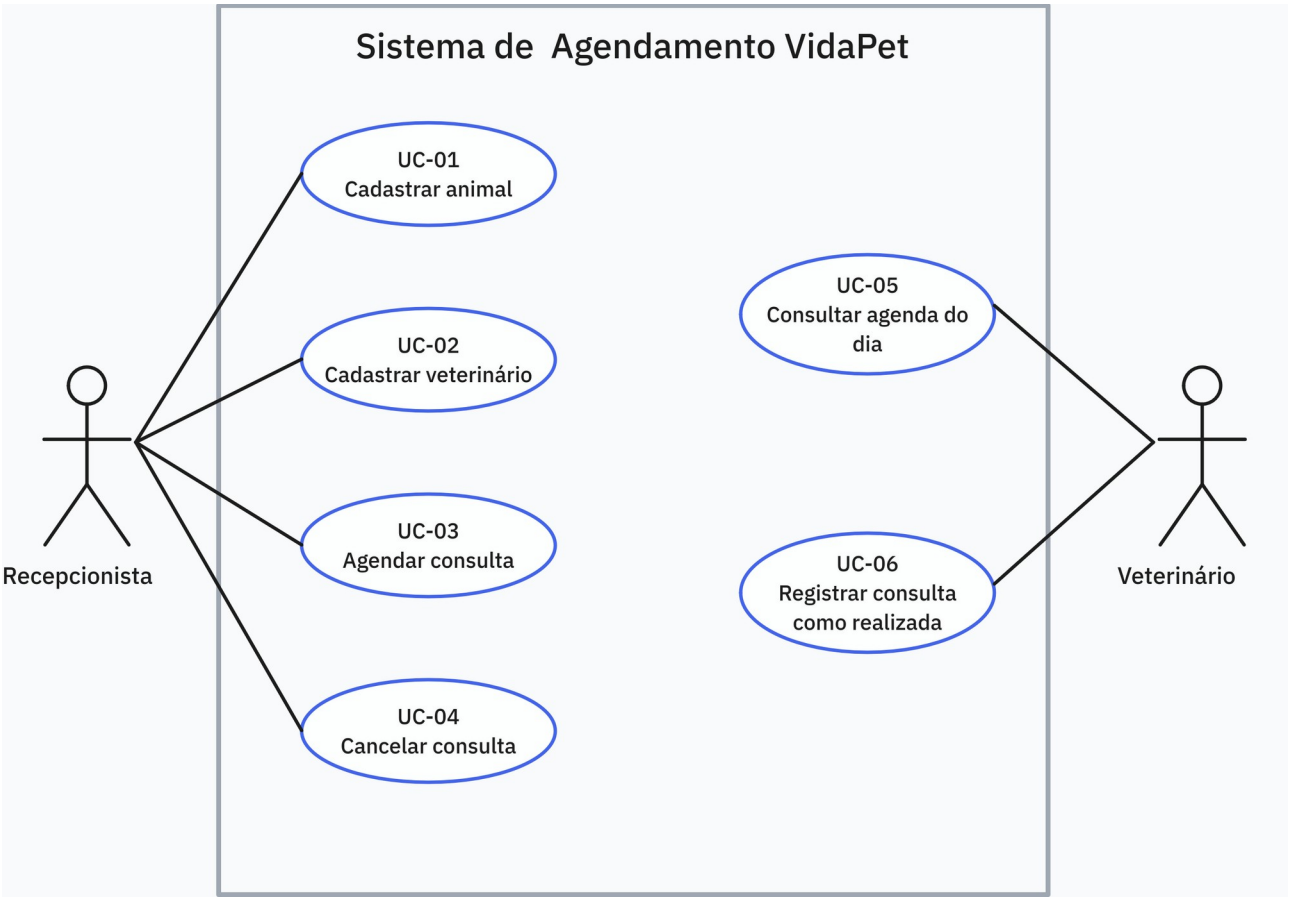
Permitir cancelamento apenas de consultas agendadas

RN-06

Permitir conclusão apenas de consultas agendadas e pelo veterinário responsável

Tarefa 2 – Diagrama UML de Casos de Uso

Diagrama UML



Especificação Textual dos Casos de Uso

UC-01 – Cadastrar Animal

Ator principal: Recepcionista.

Objetivo: Permitir o cadastro dos animais atendidos pela clínica, armazenando suas informações e os dados de contato dos tutores.

Pré-condição: A recepcionista deve ter acesso à funcionalidade de cadastro de animais.

Fluxo principal:

1. A recepcionista acessa a opção de cadastrar animal.
2. O sistema solicita as informações do animal.
3. A recepcionista informa o nome, espécie (cão ou gato), idade, nome do tutor e telefone do tutor.
4. O sistema verifica se os dados informados são válidos.
5. O sistema registra o animal.
6. O sistema apresenta uma mensagem confirmando o cadastro.

Fluxo alternativo:

- **FA-01 – Dados inválidos ou incompletos:** Caso alguma informação obrigatória não seja preenchida ou apresente um valor inválido, o sistema informa o problema e solicita a correção antes de concluir o cadastro.

Pós-condição: O animal fica cadastrado no sistema e disponível para futuros agendamentos.

UC-02 – Cadastrar Veterinário

Ator principal: Recepcionista.

Objetivo: Permitir o cadastro dos veterinários que realizam atendimentos na clínica.

Pré-condição: A recepcionista deve ter acesso à funcionalidade de cadastro de veterinários.

Fluxo principal:

1. A recepcionista acessa a opção de cadastrar veterinário.
2. O sistema solicita o nome e o número de registro profissional (CRMV).
3. A recepcionista informa os dados do veterinário.
4. O sistema verifica se os dados informados são válidos.
5. O sistema registra o veterinário.
6. O sistema apresenta uma mensagem confirmando o cadastro.

Fluxo alternativo:

- **FA-01 – Dados inválidos ou incompletos:** Caso o nome ou o CRMV não seja informado corretamente, o sistema apresenta uma mensagem de erro e solicita a correção dos dados.

Pós-condição: O veterinário fica cadastrado e disponível para receber agendamentos de consultas.

UC-03 – Agendar Consulta

Ator principal: Recepcionista.

Objetivo: Permitir o agendamento de consultas veterinárias, respeitando os horários de funcionamento da clínica e evitando conflitos na agenda dos profissionais.

Pré-condições:

- O animal deve estar cadastrado no sistema.
- O veterinário deve estar cadastrado no sistema.

Fluxo principal:

1. A recepcionista acessa a opção de agendar consulta.
2. O sistema solicita o animal, veterinário, data e horário.
3. A recepcionista informa os dados do agendamento.
4. O sistema verifica se a data escolhida não está no passado.
5. O sistema verifica se a consulta está dentro dos dias e horários de funcionamento da clínica.
6. O sistema verifica se o veterinário não possui outra consulta agendada no mesmo horário.
7. O sistema registra a consulta com a situação "Agendada".
8. O sistema apresenta uma mensagem confirmando o agendamento.

Fluxos alternativos:

- **FA-01 – Horário indisponível:** Caso o veterinário já tenha uma consulta agendada no horário informado, o sistema impede o agendamento e solicita a escolha de outro horário.
- **FA-02 – Data inválida:** Caso a data informada esteja no passado, o sistema rejeita o agendamento e solicita uma nova data.
- **FA-03 – Fora do horário de funcionamento:** Caso o agendamento seja solicitado para um sábado, domingo ou horário fora do expediente, o sistema informa que a clínica realiza consultas apenas de segunda a sexta-feira, das 8h às 18h.
- **FA-04 – Cadastro inexistente:** Caso o animal ou o veterinário não esteja cadastrado, o sistema impede o agendamento até que os dados necessários estejam disponíveis.

Pós-condição: A consulta fica registrada no sistema com a situação "Agendada" e vinculada ao animal e ao veterinário selecionados.

Regras de negócio relacionadas:

- As consultas possuem duração de 30 minutos.
- O funcionamento ocorre de segunda a sexta-feira, das 8h às 18h.
- Um veterinário não pode possuir duas consultas agendadas no mesmo horário.
- Não é permitido realizar agendamentos em datas passadas.

UC-04 – Cancelar Consulta

Ator principal: Recepcionista.

Objetivo: Permitir o cancelamento de consultas previamente agendadas, respeitando a situação atual do atendimento.

Pré-condições:

- A consulta deve estar registrada no sistema.
- A consulta deve apresentar a situação "Agendada".

Fluxo principal:

1. A recepcionista acessa a opção de cancelar consulta.
2. O sistema apresenta as consultas disponíveis para consulta e seleção.
3. A recepcionista seleciona a consulta que deseja cancelar.
4. O sistema verifica a situação atual da consulta.
5. Caso esteja como "Agendada", o sistema permite o cancelamento.
6. O sistema altera a situação da consulta para "Cancelada".
7. O sistema apresenta uma mensagem confirmando o cancelamento.

Fluxos alternativos:

- **FA-01 – Consulta já realizada:** Caso a consulta esteja com a situação "Realizada", o sistema impede o cancelamento e informa que a operação não é permitida.
- **FA-02 – Consulta já cancelada:** Caso a consulta já esteja com a situação "Cancelada", o sistema informa que não é possível realizar novamente o cancelamento.
- **FA-03 – Consulta inexistente:** Caso a consulta selecionada não seja encontrada, o sistema informa que não foi possível localizar o agendamento.

Pós-condição: A consulta passa a apresentar a situação "Cancelada", mantendo seu registro no sistema para consulta posterior.

UC-05 – Consultar Agenda do Dia

Ator principal: Veterinário.

Objetivo: Permitir que o veterinário visualize as consultas associadas a ele em determinado dia, facilitando a organização dos atendimentos.

Pré-condições:

- O veterinário deve estar cadastrado no sistema.
- O sistema deve conseguir identificar o veterinário responsável pela agenda consultada.

Fluxo principal:

1. O veterinário acessa a opção de consultar agenda do dia.
2. O sistema identifica o veterinário e a data da consulta.
3. O sistema procura as consultas associadas ao profissional naquela data.
4. O sistema organiza as consultas encontradas.
5. O sistema apresenta os horários, os animais e as situações das consultas.
6. O veterinário visualiza sua agenda diária.

Fluxos alternativos:

- **FA-01 – Agenda vazia:** Caso não existam consultas registradas para o veterinário na data selecionada, o sistema apresenta uma mensagem informando que não há atendimentos registrados para aquele dia.
- **FA-02 – Veterinário não identificado:** Caso o sistema não consiga identificar o profissional responsável pela agenda, a consulta não é apresentada até que o veterinário seja identificado corretamente.

Pós-condição: A agenda é exibida ao veterinário, sem alterar os dados ou as situações das consultas.

UC-06 – Registrar Consulta como Realizada

Ator principal: Veterinário responsável pela consulta.

Objetivo: Permitir que o veterinário registre a conclusão do atendimento, alterando a situação da consulta e adicionando as observações realizadas durante o atendimento.

Pré-condições:

- A consulta deve estar registrada no sistema.
- A consulta deve apresentar a situação "Agendada".
- O veterinário responsável pelo registro deve ser o mesmo veterinário associado à consulta.

Fluxo principal:

1. O veterinário acessa sua agenda de consultas.
2. O sistema apresenta os atendimentos vinculados ao profissional.
3. O veterinário seleciona uma consulta agendada.
4. O veterinário solicita o registro da consulta como realizada.
5. O sistema permite o preenchimento das observações do atendimento.

6. O veterinário informa as observações.
7. O sistema verifica se a consulta está agendada e se o profissional é o veterinário responsável.
8. O sistema atualiza a situação da consulta para "Realizada" e registra as observações.
9. O sistema apresenta uma mensagem confirmando a conclusão do atendimento.

Fluxos alternativos:

- **FA-01 – Consulta cancelada:** Caso a consulta esteja com a situação "Cancelada", o sistema impede sua conclusão e informa que uma consulta cancelada não pode ser marcada como realizada.
- **FA-02 – Consulta já realizada:** Caso a consulta já esteja com a situação "Realizada", o sistema informa que o atendimento já foi concluído e impede um novo registro de conclusão.
- **FA-03 – Veterinário não autorizado:** Caso o profissional que está tentando registrar a conclusão não seja o veterinário responsável pela consulta, o sistema bloqueia a operação.

Pós-condição: A consulta passa a apresentar a situação "Realizada", com as observações do atendimento registradas no sistema.

Regras de negócio relacionadas:

- Somente consultas com situação "Agendada" podem ser marcadas como realizadas.
- Apenas o veterinário responsável pela consulta pode registrar sua conclusão.

Tarefa 3 – Configuração do Ambiente de Desenvolvimento C#

Ambiente configurado com C#

felipe_meira@Notebook-Felipe-Meira:~$ dotnet –list-sdks

10.0.114 [/usr/lib/dotnet/sdk]

felipe_meira@Notebook-Felipe-Meira:~$ dotnet --version

10.0.114

felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco$ dotnet new console --framework net10.0 -n VidaPet.TP1

O modelo "Aplicativo do Console" foi criado com êxito.

Processando ações pós-criação...

Restaurando /home/felipe_meira/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1/VidaPet.TP1.csproj:

A restauração foi bem-sucedida.

felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco$ cd VidaPet.TP1

felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1$ dotnet build

Restauração concluída (0,8s)

VidaPet.TP1 net10.0 êxito (0,6s) → bin/Debug/net10.0/VidaPet.TP1.dll

Construir êxito em 2,1s

felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1$ dotnet run

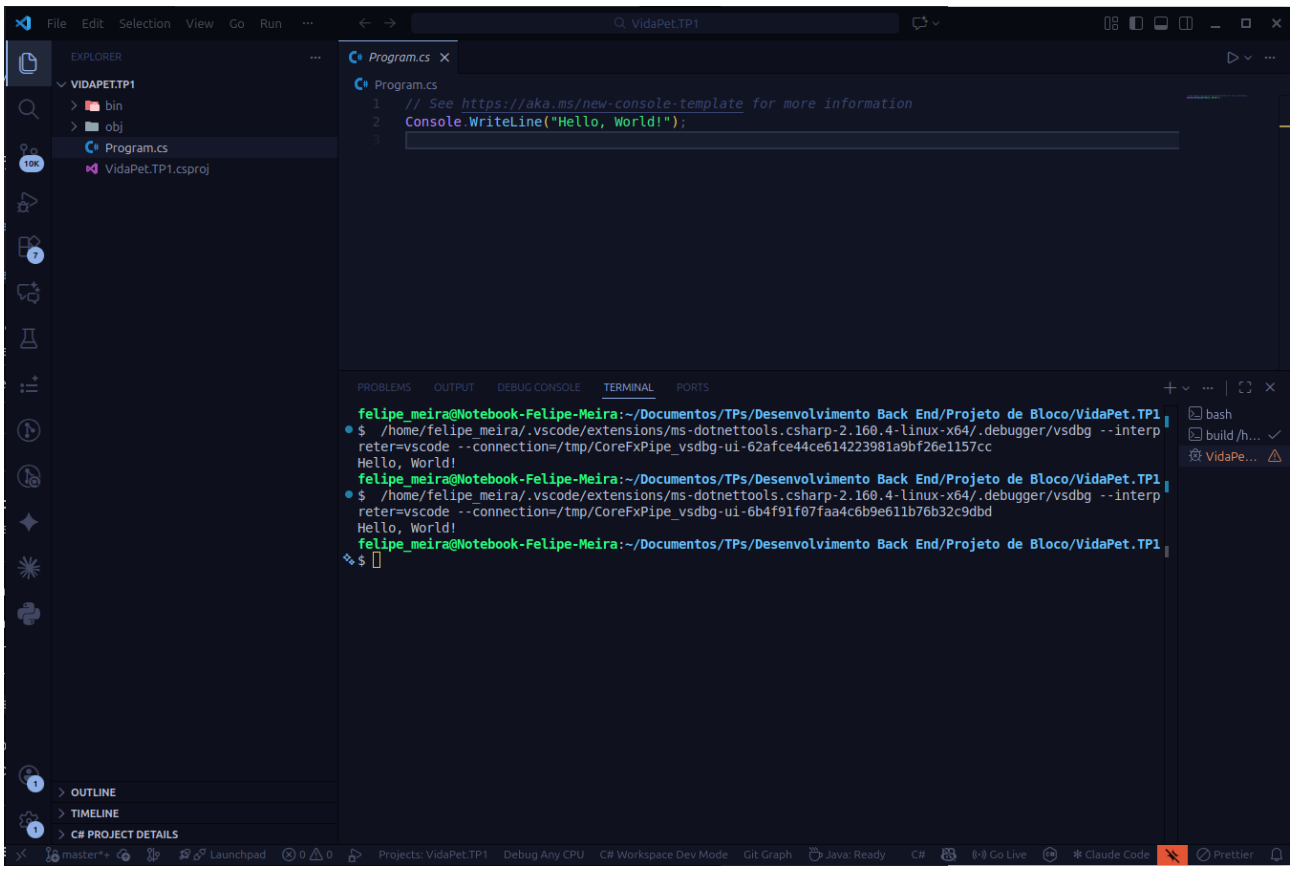
Hello, World!

```
felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco$ dotnet new console --framework net10.0 -n VidaPet.TP1
O modelo "Aplicativo do Console" foi criado com êxito.

Processando ações pós-criação...
Restaurando /home/felipe_meira/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1/VidaPet.TP1.csproj:
A restauração foi bem-sucedida.


felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco$ cd VidaPet.TP1
felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1$ dotnet build
Restauração concluída (0,8s)
  VidaPet.TP1 net10.0 êxito (0,6s) → bin/Debug/net10.0/VidaPet.TP1.dll

Construir êxito em 2,1s
felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1$ dotnet run
felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1$ dotnet run
Hello, World!
felipe_meira@Notebook-Felipe-Meira:~/Documentos/TPs/Desenvolvimento Back End/Projeto de Bloco/VidaPet.TP1$
```



Tarefa 4 – Programa Standalone inicial em C#

```
using System;

class Program
{
	static void Main(string[] args)
	{
		Console.WriteLine("Simulador de horários VidaPet");
		Console.WriteLine();

		Console.Write("Dia da semana (1 = segunda-feira, 2 = terça, 3 = quarta, 4 = quinta,
		5 = sexta, 6 = sábado, 7 = domingo): ");
		bool diaEhInteiro = int.TryParse(Console.ReadLine(), out int diaSemana);

		Console.Write("Hora (0 a 23): ");
		bool horaEhInteira = int.TryParse(Console.ReadLine(), out int hora);

		Console.Write("Minuto (0 a 59): ");
		bool minutoEhInteiro = int.TryParse(Console.ReadLine(), out int minuto);

		bool entradasInteiras = diaEhInteiro && horaEhInteira && minutoEhInteiro;

		if (!entradasInteiras)
		{
			Console.WriteLine("Entrada inválida. Digite números inteiros para dia, hora
			e minuto.");
			return;
		}

		bool diaPermitido = diaSemana >= 1 && diaSemana <= 5;
		bool horaMinutoValidos = hora >= 0 && hora <= 23 && minuto >= 0 && minuto <=
		59;

		int inicioConsulta = hora * 60 + minuto;
		int fimConsulta = inicioConsulta + 30;

		bool horarioPermitido = diaPermitido
		&& horaMinutoValidos
		&& inicioConsulta >= 8 * 60
		&& fimConsulta <= 18 * 60;
		Console.WriteLine();

		if (horarioPermitido)
		{
```

```
                    Console.WriteLine("Horário permitido");
                    Console.WriteLine("A consulta tem duração de 30 minutos.");
                }
                else
                {
                    Console.WriteLine("Horário não permitido");

                if (!diaPermitido)
                {
                    Console.WriteLine("O atendimento ocorre somente de segunda a sexta-f
                    eira (dias de 1 a 5).");
                }
                else if (!horaMinutoValidos)
                {
                    Console.WriteLine("Informe uma hora entre 0 e 23 e um minuto entre 0 e
                    59.");
                }
                else if (hora <= 7)
                {
                    Console.WriteLine("O atendimento começa às 08h00.");
                }
                else
                {
                    Console.WriteLine("A consulta deve terminar até as 18h00. O último início
                    permitido é às 17h30.");
                }
            }
        }
    }
}
```

Tarefa 5 – Justificativa do escopo definido

Objetivo e definição do escopo

O objetivo foi realizar a análise inicial e a modelagem do Sistema de Agendamento da Clínica Veterinária VidaPet, buscando compreender o problema apresentado e definir os requisitos necessários para o desenvolvimento do sistema.

Como a clínica atualmente utiliza um caderno para organizar as consultas, podem ocorrer problemas como agendamentos duplicados e falta de controle dos atendimentos causando conflito e insatisfação dos clientes. Por isso, nesta primeira etapa, o foco foi entender essas dificuldades, identificar as principais funcionalidades e organizar as regras de negócio antes de iniciar o desenvolvimento completo da aplicação.

Funcionalidades incluídas no TP1

Nesta entrega, foram levantados os requisitos funcionais, requisitos não funcionais e regras de negócio, identificadas as principais entidades do sistema (Animal, Veterinário e Consulta) e elaborado um diagrama UML com seis casos de uso, representando as interações da recepcionista e do veterinário com o sistema.

Também foi desenvolvido um programa standalone inicial em C#, utilizando o .NET 10, também utilizei o Visual Studio Code pois estou no Linux e a extensão C# Dev Kit. O programa funciona como um simulador de horários para consultas veterinárias. Nele, o usuário informa o dia da semana, a hora e os minutos desejados para o atendimento.

A partir dessas informações, o programa utiliza variáveis, operadores lógicos e estruturas condicionais para verificar se a consulta está dentro dos dias e horários de funcionamento da clínica, considerando também a duração de 30 minutos.

Dessa forma, foi possível aplicar algumas das regras de negócio do sistema sem a necessidade de desenvolver todas as funcionalidades nesta primeira etapa.

Funcionalidades deixadas para as próximas etapas

Algumas funcionalidades foram deixadas para as próximas etapas do projeto, como o cadastro completo de animais e veterinários, o armazenamento das consultas, a identificação de conflitos entre agendamentos e a atualização das situações das consultas.

Também ficaram para etapas futuras a implementação das classes de domínio utilizando orientação a objetos, a persistência em banco de dados, a criação de uma API e o controle de autenticação dos usuários.

Essa divisão foi realizada porque o objetivo do TP1 é trabalhar inicialmente com análise, requisitos, modelagem UML e fundamentos de programação.

Assim, o programa desenvolvido nesta etapa não representa o sistema completo da clínica, mas uma implementação inicial de parte das regras de negócio, que poderão ser aproveitadas e ampliadas ao longo do projeto.

Suposições e decisões adotadas

Durante a interpretação da especificação, foi necessário considerar algumas situações que não foram detalhadas completamente e fiquei em dúvida.

1. Horário de encerramento das consultas: Foi considerado que uma consulta deve terminar até as 18h, horário de fechamento da clínica. Como cada consulta dura 30 minutos, o último horário permitido para início é 17h30.

2. Validação dos dias da semana: Para simplificar o programa inicial, os dias foram representados por números inteiros de 1 a 7, sendo 1 para segunda-feira e 7 para domingo. Apenas os números de 1 a 5 são aceitos para agendamentos.

3. Validação dos horários: O programa permite informar horas entre 0 e 23 e minutos entre 0 e 59, mas somente considera permitido um atendimento que comece a partir das 8h e termine até as 18h.

4. Histórico de consultas: Foi considerada, para as próximas etapas, a possibilidade de manter as consultas canceladas registradas no sistema, alterando apenas sua situação, para preservar o histórico de atendimentos.

5. Validação de conflitos: Foi considerado que os agendamentos não deverão apresentar sobreposição de horários para o mesmo veterinário. Essa validação será desenvolvida nas próximas etapas, quando o sistema possuir o controle das consultas registradas.

6. Horários quebrados: O programa permite iniciar uma consulta em qualquer minuto, como 8h15min ou 10h47min, o tempo de 30 minutos não obriga os inícios a serem apenas às horas cheias ou às meias horas. Precisa ter essa especificação como RF? Ou mantém nesse formato?

Essas decisões ajudam a estabelecer critérios para o funcionamento do sistema e poderão ser revisadas conforme as orientações.